

IT-359 Group Project

Red Team Recon Tool
Automated Network Reconnaissance & Vulnerability Assessment

Authors: Dylan Voss & Grant Gollinger
Course: IT-359
Illinois State University
Spring 2026

Tool: recon_tool.sh — Modular Bash-based red team recon script
Technologies: Bash 4.0+, Nmap 7.80+, Claude API (Haiku), curl, jq, grep, awk, sed
Features: Host discovery, TCP/UDP scanning, OS detection, vuln scripting, AI threat analysis, structured reporting
Test Environment: Hack The Box — Starting Point Tier 0 (Meow)

1. Introduction & Purpose

Network security is one of the most pressing challenges facing modern organizations. Penetration testing — the practice of authorized, simulated attacks against a system or network to identify vulnerabilities before malicious actors can exploit them — is a cornerstone of proactive defense. Yet the reconnaissance phase that opens every penetration test is inherently tedious: security professionals must discover live hosts, scan thousands of ports, enumerate running services, identify version-specific risks, and compile findings into a coherent report. Performed manually, this work is slow, error-prone, and difficult to reproduce consistently across engagements.

The **Red Team Recon Tool**, developed by Dylan Voss and Grant Gollinger for IT-359, directly addresses this problem. Built in Bash and powered by Nmap, the tool automates the full early-stage reconnaissance pipeline — from host discovery and full TCP/UDP port scanning, through service enumeration, OS fingerprinting, optional vulnerability scripting, and risk identification — culminating in a structured report with per-port mitigation recommendations and, optionally, an AI-powered threat assessment generated by the Claude API. The result is a single command that replaces hours of manual work with a consistent, repeatable, documented workflow.

1.1 Project Objectives

The tool was designed around five core objectives:

- Automate the full network reconnaissance workflow — host discovery through report generation — in a single executable Bash script that requires no configuration beyond specifying a target.
- Demonstrate practical penetration testing methodology by implementing the exact scanning phases used in real engagements: ping sweeps, SYN scans, service enumeration, OS fingerprinting, and vulnerability scripting.
- Surface actionable security intelligence by cross-referencing scan results against a curated 33-entry risk database and generating per-port, targeted mitigation guidance based on what was actually found.
- Integrate AI-driven analysis by connecting scan findings to the Anthropic Claude API, enabling natural-language threat assessments that contextualize risks, suggest attack vectors, and prioritize remediation.
- Operate within strict ethical and legal boundaries, with the tool designed to require explicit operator-specified targets and restricted entirely to reconnaissance — no exploitation capabilities are included.

1.2 Ethical & Legal Considerations

This project was conducted strictly for educational purposes under the supervision of IT-359 course requirements. All scanning and testing was performed exclusively against systems owned by the project team or machines on the Hack The Box platform, which provides explicit written authorization for security testing as part of its service. No public networks, production systems, or unauthorized hosts were targeted at any point.

The README explicitly warns users that unauthorized scanning may violate the **Computer Fraud and Abuse Act (CFAA)** and equivalent state and international laws. The tool enforces this posture by design: it requires the operator to explicitly specify a target via the **-t** flag and includes no autonomous target discovery or unsolicited scanning capability. The tool performs reconnaissance and enumeration only — it contains no exploitation code.

2. Technical Implementation

The tool is implemented as a single modular Bash script (**recon_tool.sh**) organized into discrete functions corresponding to each phase of the reconnaissance pipeline. It is invoked with a required target flag (**-t**) and several optional flags that activate additional scanning modules. All output is written to a timestamped directory under *./reports/*, keeping results from each run cleanly isolated.

2.1 Tools & Technologies

Tool / Technology	Version / Requirement	Role
Bash	4.0+ (associative arrays)	Primary scripting language; orchestrates all phases
Nmap	7.80+	Host discovery, port scanning, service/OS/vuln scanning
curl	Any recent	HTTP client for Anthropic Claude API calls (AI mode)
jq	Any recent	JSON construction and parsing for Claude API integration
grep / awk / sed	Standard GNU	Output parsing, field extraction, string manipulation
Claude API (Haiku)	claude-haiku-4-5	AI-powered threat assessment and attack path analysis
Linux (Ubuntu / Kali)	22.04 / 24.04	Primary OS; macOS supported with reduced functionality

2.2 Command-Line Interface

The script uses Bash **getopts** for argument parsing, providing a clean Unix-style interface:

```
Usage: sudo ./src/recon_tool.sh -t [options] -t Target IP or subnet (required) e.g.
10.129.1.17 or 192.168.1.0/24 -o Output directory default: ./reports -u Enable UDP scan on
high-risk ports -v Enable vulnerability script scanning (nmap --script vuln) -a Enable AI
threat analysis (requires ANTHROPIC_API_KEY) -h Show help message # Basic scan sudo
./src/recon_tool.sh -t 10.129.1.17 # Full scan: UDP + vuln scripts + AI analysis export
ANTHROPIC_API_KEY=sk-ant-... sudo ./src/recon_tool.sh -t 10.10.10.5 -u -v -a
```

Note: sudo is required for SYN scanning (**-sS**) and OS detection (**-O**). Without root privileges the tool falls back to a TCP connect scan and skips OS fingerprinting.

2.3 Phase-by-Phase Implementation

Phase 1 — Host Discovery

Target mode is determined by checking for a CIDR prefix. For **subnet targets**, an **nmap -sn** ping sweep sends ICMP echo requests and TCP/ARP probes to identify live hosts, with results parsed by grep and awk into *live_hosts.txt*. For **single-host targets**, the ping sweep is skipped entirely

and Nmap's **-Pn** flag is used to treat the target as always up — a critical accommodation for Hack The Box machines and enterprise firewalls that silently drop ICMP, which would otherwise cause the host to appear offline.

Phase 2 — TCP Port Scanning

A full SYN stealth scan (**nmap -sS -p- --open -T4 --min-rate 1000**) covers all 65,535 TCP ports. The T4 timing template and 1,000 packet minimum rate balance thoroughness against scan duration — critical in lab environments with time constraints. The **--open** flag suppresses closed and filtered ports, keeping output focused. Results are written in both XML (*raw_scan.xml*) and grepable (*ports.gnmap*) formats for downstream processing.

Phase 3 — Service Enumeration & OS Detection

For hosts with open ports, **nmap -sV -sC -O** performs three tasks simultaneously: service version detection (**-sV**) identifies the software name and version on each open port (e.g., *OpenSSH 8.2p1*, *Apache httpd 2.4.51*); default NSE scripts (**-sC**) run targeted checks such as anonymous FTP login, SSL certificate inspection, and SMB share enumeration; and OS fingerprinting (**-O**) attempts to identify the target operating system from TCP/IP stack behavior. OS detection results are written to *os_detection.txt*.

Phase 4 — Optional UDP Scanning

When invoked with **-u**, the tool performs a targeted UDP scan against 12 high-risk ports: SNMP (161), DNS (53), TFTP (69), NTP (123), NetBIOS (137/138), DHCP (67/68), IKE/IPSec (500), mDNS (5353), SSDP (1900), and RPC (111). UDP scanning is intentionally scoped to these ports rather than all 65,535 because UDP scans are substantially slower and less reliable than TCP — a full UDP sweep would take hours. Results are saved to *udp_scan.txt* and *udp_ports.gnmap*.

Phase 5 — Vulnerability Script Scanning

The optional **-v** flag triggers **nmap --script vuln** against discovered hosts, running Nmap's full vulnerability detection script library. This includes checks for known CVEs, misconfigured services, and exploitable conditions such as MS17-010 (EternalBlue), Heartbleed, and anonymous access vulnerabilities. After the scan, the script automatically extracts CVE identifiers from the output using `grep` and writes them to *cves_found.txt* for quick triage. Full results are preserved in *vuln_scan.txt* and *vuln_scan.xml*.

Phase 6 — Risk Identification

The script maintains a Bash associative array (**declare -A RISKY_PORTS**) mapping 33 well-known port numbers to plain-English risk descriptions. An `awk` state-machine parser accurately associates each risky open port with the correct host across multi-host subnet scans — a non-trivial parsing challenge in Nmap's text output format. Matches are written to *risks.txt* with

host attribution and also tracked in *risky_ports_found.txt* for use in the targeted mitigation step.

The 33-entry risk database spans five categories:

Category	Ports Covered
Unencrypted legacy protocols	FTP (21), Telnet (23), rexec (512), rlogin (513), rsh (514)
Exposed database services	MySQL (3306), MSSQL (1433), Oracle (1521), PostgreSQL (5432), MongoDB (27017), Redis (6379), Elasticsearch (9200)
Windows attack surface	MS-RPC (135), NetBIOS (137/139), SMB (445), RDP (3389)
Misconfiguration risks	SNMP (161), NFS (2049), VNC (5900), HTTP (80/8080), LDAP (389)
Backdoor / C2 indicators	Port 4444 (Metasploit default listener)

Phase 7 — AI Threat Analysis

The most distinctive feature of the updated tool is its integration with the **Anthropic Claude API** (*claude-haiku-4-5-20251001*). When invoked with **-a**, the script compiles the scan findings — open ports, service versions, OS detection results, identified risks, and any extracted CVEs — into a structured prompt and sends it to the Claude API via curl. The jq utility is used to safely construct the JSON request body, handling special characters in scan output that would otherwise break JSON encoding.

The API returns a structured threat assessment covering five areas:

- Threat Summary — Overall security posture of the target based on the scan findings
- Critical Findings — Highest-severity risks with CVE identifiers where applicable
- Suggested Attack Vectors — Specific next steps with tool names and example commands a tester could run
- Prioritized Mitigations — Remediation guidance ordered by severity and exploitability
- Additional Recon — Further enumeration tools and techniques recommended for deeper investigation

The AI analysis is saved to *ai_analysis.txt* and appended to the final report. This transforms the tool from a data-collection utility into an intelligent assistant that interprets findings and guides the next steps of an engagement — functionality that would normally require a senior analyst's review.

Phase 8 — Report Generation

The final report (*report.txt*) consolidates all findings into a structured, human-readable document. Critically, the mitigation section is **dynamic** rather than static boilerplate: the script reads *risky_ports_found.txt* and generates per-port remediation advice specific to what was actually

discovered on the target. A scan that finds only Telnet will produce Telnet-specific guidance; one that finds SMB and Redis will produce guidance tailored to those services. This per-port approach is significantly more useful to an analyst than generic recommendations.

2.4 Output Directory Structure

```
reports/ ■■■ 20260416_142654_10.129.1.17/ ■■■ report.txt ← Final structured report ■■■
live_hosts.txt ← Discovered live hosts ■■■ raw_scan.xml ← Full TCP port scan (XML) ■■■
ports.gnmap ← TCP port scan (grepable) ■■■ services.xml/txt ← Service enumeration
results ■■■ os_detection.txt ← OS fingerprint summary ■■■ risks.txt ← Identified risk
log ■■■ risky_ports_found.txt ← Port numbers that triggered risks ■■■
udp_scan.txt/gnmap ← UDP results (if -u used) ■■■ vuln_scan.txt/xml ← Vuln script
results (if -v used) ■■■ cves_found.txt ← Extracted CVEs (if -v used) ■■■
ai_analysis.txt ← Claude threat assessment (if -a used)
```

2.5 Sample Output — Hack The Box: Meow

The tool was validated against **Meow**, the first machine in Hack The Box's Starting Point Tier 0. Meow is an intentionally vulnerable Linux machine built around an open, unauthenticated Telnet service — a textbook example of the unencrypted legacy protocol risk category in the tool's risk database. The scan was run in single-host mode against target IP **10.129.1.17** over the HTB VPN, completing in under 21 seconds.

```
===== IT-359 NETWORK
RECONNAISSANCE & VULNERABILITY REPORT Generated: Thu Apr 16 02:27:23 PM CDT 2026 Target:
10.129.1.17 Scan Mode: single (single host, ping disabled (-Pn)) Authors: Dylan Voss &
Grant Gollinger ===== SECTION 1:
LIVE HOSTS ----- Total live hosts
found: 1 10.129.1.17 SECTION 2: SERVICE ENUMERATION
----- Nmap scan report for
10.129.1.17 Host is up (0.0094s latency). Not shown: 999 closed tcp ports (reset) PORT
STATE SERVICE VERSION 23/tcp open telnet Linux telnetd Service Info: OS: Linux; CPE:
cpe:/o:linux:linux_kernel Nmap done: 1 IP address (1 host up) scanned in 20.52 seconds
SECTION 3: IDENTIFIED RISKS -----
[RISK] Port 23 - Telnet - Unencrypted remote access, credential sniffing risk Affected:
10.129.1.17 SECTION 4: MITIGATION RECOMMENDATIONS
----- [Port 23 - Telnet] Disable
Telnet immediately. Replace with SSH for all remote access. If Telnet must remain,
restrict via firewall to authorized IPs only.
===== END OF REPORT
=====
```

The risk identification phase correctly matched port 23 against the Telnet entry in the *RISKY_PORTS* dictionary and attributed it to the correct host. The mitigation output is port-specific — not generic boilerplate — because the script reads *risky_ports_found.txt* and generates targeted guidance for only the ports that were actually discovered. On Meow, Telnet accepts a root login with no password, which is exactly the exploitation path the HTB machine is designed to demonstrate. The tool surfaces this vulnerability in a single automated pass, validating its core reconnaissance design.

3. Justification & Analysis

3.1 Why Bash?

Bash was chosen as the implementation language for three concrete reasons. First, it is natively present on every Linux distribution — including Kali Linux, the industry-standard penetration testing OS — which means the tool has zero runtime dependencies beyond Nmap itself for its core functionality. Second, Bash is purpose-built for orchestrating command-line tools: invoking Nmap, piping output through `grep` and `awk`, writing files, and checking exit codes are all idiomatic Bash operations achievable in far fewer lines than equivalent Python using subprocess calls. Third, Bash scripting is a skill used daily by practicing security engineers; building this project in Bash gives the team direct, transferable experience with the same tooling used in real engagements.

The one significant Bash constraint encountered was associative arrays, which require Bash 4.0+. macOS ships with Bash 3.2 by default, which is why the README notes macOS as supported 'with caveats.' This was a deliberate trade-off: associative arrays enabled clean $O(1)$ port-to-risk lookups, which was worth the minor compatibility restriction given the Linux-first target environment.

3.2 Why Nmap?

Nmap is the most widely deployed network scanner in the security industry and the de facto standard for penetration testing reconnaissance. Its support for multiple output formats (XML, greppable, normal text) makes it ideal for scripted pipelines — the XML output in particular enables structured post-processing that `grep`-based parsing of plain text cannot reliably achieve. The Nmap Scripting Engine (NSE) eliminates the need for separate tools: a single `-sC` flag runs checks that would otherwise require standalone utilities for anonymous FTP detection, SSL inspection, SMB enumeration, and HTTP header analysis.

The vulnerability scanning phase (`--script vuln`) extends Nmap's capability further, checking for known CVEs and exploitable conditions in a single pass. Alternatives like `masscan` are faster for pure port discovery at internet scale but lack version detection, scripting, and OS fingerprinting — making Nmap the only tool that could consolidate all required reconnaissance phases without adding additional dependencies.

3.3 Why the Claude API for AI Integration?

The decision to integrate AI analysis via the Anthropic Claude API was driven by the fundamental difference between what scripted logic does well and what language models do well. Rule-based logic — port matching, risk flagging, structured output — is better handled deterministically by the script itself. But *interpreting* findings, contextualizing risks, suggesting specific attack vectors with tool names and commands, and explaining why a particular service configuration is dangerous — these are tasks that benefit enormously from a language model's broad security knowledge.

The **claude-haiku-4-5-20251001** model was selected specifically because it offers strong instruction-following and security domain knowledge at low latency and cost — important properties for a tool that may be run repeatedly during an engagement. The integration uses curl and jq rather than a Python SDK, keeping the tool's dependency footprint minimal and consistent with its Bash-first architecture. The API key is user-supplied via environment variable, ensuring the tool never stores credentials.

3.4 Design Decisions & Trade-offs

Decision	Rationale	Trade-off
SYN scan (-sS) over TCP connect	Faster; less likely to be logged by services	Requires root/sudo
-p- across all 65,535 ports	Catches services on non-standard ports	Slower than top-1000 scan
T4 + --min-rate 1000	Practical speed for lab/HTB environments	May cause packet loss on slow networks
-Pn for single hosts	Supports firewalled and HTB targets	Cannot confirm host is alive
Scoped UDP (12 ports only)	Avoids hours-long full UDP sweep	Misses UDP services on non-standard ports
Dynamic per-port mitigations	More actionable than generic recommendations	Requires risky_ports_found.txt to be accurate
jq for JSON construction	Safe handling of special chars in scan output	Adds jq as dependency for AI mode
Timestamped output directories	Preserves full history of all scan runs	Accumulates disk usage over time

3.5 Validation & Testing

The tool was tested on Hack The Box's Meow machine, an authorized lab environment that replicates real-world conditions — including ICMP-blocking firewalls that require the **-Pn** flag to function correctly. The Meow test confirmed that all core phases operate correctly end-to-end: the single-host mode correctly bypassed ping discovery, the full port scan identified port 23 as open, service enumeration confirmed Linux telnetd, the risk identification phase matched port 23 against the Telnet entry with correct host attribution, and the report generator produced targeted Telnet-specific mitigation guidance. HTB's legal authorization framework also allowed the team to test against a realistic network environment without ethical or legal concerns.

4. Conclusion

The Red Team Recon Tool represents a significant step beyond a basic course project. What began as an automation exercise evolved into a multi-phase, production-oriented security tool that integrates real scanning technology, a curated security knowledge base, and AI-driven threat analysis into a single, portable Bash script. The tool's architecture closely mirrors the reconnaissance workflow used in professional penetration testing engagements, and its validation on a live Hack The Box machine confirms that it functions correctly in conditions that replicate real-world network environments.

Most importantly, the project demonstrates a mature understanding of *why* each design decision was made: why Bash over Python, why SYN scanning over TCP connect, why a scoped UDP scan rather than a full sweep, why the Claude API for the AI layer. Each choice reflects a deliberate trade-off analysis grounded in how penetration testing is actually practiced, not just what was easiest to implement.

4.1 Key Outcomes

- Delivered a complete 8-phase reconnaissance pipeline — host discovery, TCP scanning, service enumeration, OS detection, optional UDP scanning, optional vuln scripting, risk identification, and report generation — in a single modular Bash script.
- Implemented intelligent target-mode detection with appropriate Nmap flags for each case, including -Pn support for firewalled and HTB targets that block ICMP.
- Built a 33-entry risk database with an awk state-machine parser that accurately maps each risk to the correct host across multi-host subnet scans.
- Integrated the Anthropic Claude API to generate structured AI threat assessments covering attack vectors, CVE-specific findings, and prioritized remediation guidance.
- Produced dynamic, per-port mitigation recommendations based on actual scan findings rather than static boilerplate, significantly improving report usefulness.
- Validated the complete workflow on Hack The Box's Meow machine under authorized conditions, confirming correct operation in a realistic, firewalled network environment.

4.2 Limitations

Several limitations remain within the scope of a course project. Risk identification is port-number based rather than version-based, so a fully-patched service on a flagged port will still generate a risk entry — false positives are possible. The vulnerability scripting phase (**--script vuln**) can be slow and noisy, occasionally generating false positives in Nmap's own output. The tool has only been validated on a single HTB machine; broader testing across diverse network configurations would be needed before professional use. Finally, the AI analysis quality depends on the

completeness of the scan findings passed to the API — sparse results produce less actionable assessments.

4.3 Future Improvements

- Version-based CVE correlation: Cross-reference detected service versions against a local searchsploit or NVD database to replace port-level risk flags with specific, confirmed vulnerability matches.
- Expanded AI context: Pass OS detection results, extracted CVEs, and vuln script output to the Claude API alongside the risk log, enabling richer, more specific threat assessments.
- HTML/PDF report output: Generate a formatted, visually structured report alongside the plain-text output for client-facing deliverables.
- Authenticated scanning: Add optional credential-based scanning modes (SSH, SMB, SNMP) to support internal assessments where credentials are provided in scope.
- Delta reporting: Compare results across consecutive scan runs to surface newly opened ports or changed service versions, enabling the tool to function as a continuous monitoring layer.